

SAGEVM

Definitive Systems Assembly Guide

Version	0.9.8 (LTS)
Organization	Night-Traders-Dev
Targets	SVM (Stack) / SRVM (RV64)
Document	Official Technical Manual

A comprehensive systems developer's manual for the Sage General Virtual Machine (SGVM). This volume provides a deep technical dive into software machine theory, dual-architecture execution, and advanced systems engineering patterns within the SageLang ecosystem.

Sage is not just a language; it is a full-stack vision for secure, portable, and self-hosted computing. The ecosystem bridges the gap between high-level logic and native metal.

To understand SageVM assembly, one must master the three pillars of the runtime environment:

- **SageLang:** The source language. Optimized for human developer ergonomics, indentation-based syntax, and strong typing.
- **SageVM:** The virtual infrastructure. Comprising the sgvmc compiler and sgvm interpreter, written entirely in pure Sage.
- **MetalVM:** The hardware interface. A high-performance C-based execution engine that provides JIT, GC, and native acceleration.

The interaction between these layers defines the developer experience. A systems programmer might write high-level Sage for application logic, then drop into SVM or SRVM assembly for performance-critical kernel drivers or graphics pipelines.

1.1 The Strategy of Self-Hosting

Traditional languages rely on complex C/C++ toolchains to compile themselves. SageVM breaks this dependency. By being written in Sage, the VM can compile itself, run itself, and even evolve itself as long as a basic interpreter is present. This "meta-circular" design ensures that the entire stack is auditable and secure from the bottom up.

2. The Theory of Virtual Machines

A Virtual Machine (VM) is an architectural abstraction that emulates a computer system. While physical CPUs like x86 or ARM execute instructions directly on silicon, a VM executes a virtual instruction set (Bytecode) via a software loop.

2.1 Hardware vs. Software Machines

Silicon-based hardware is fixed. Adding a new instruction requires a decade of R&D; and billions in fabrication costs. Software machines are malleable. We can add a "GPU_DRAW" opcode to SageVM in a single afternoon.

Metric	Hardware Machine	Software Machine (VM)
Implementation	Transistors / VLSI	Interpreter / JIT
Instruction Set	Immutable (ISA)	Software Defined
Update Cycle	Years (Generation)	Minutes (Deploy)
Sandboxing	MMU / Ring 0	Instruction-Level Capability

2.2 The Execution Loop

Every VM revolves around a core "Fetch-Decode-Execute" loop. The VM reads a byte (Fetch), determines what it does (Decode), and calls the corresponding logic in the host engine (Execute). In SageVM, this loop is optimized via "threaded dispatch" to minimize branch mispredictions.

3. Bytecode and Assembly: The Human Interface

If SageLang is the language of the mind, and machine code is the language of the electron, then **Bytecode** is the language of the Virtual Machine.

3.1 The Nature of Bytecode

Bytecode consists of numeric opcodes. For example, the number 0x0F might represent "ADD". To a human, 0x0F is meaningless. To the VM, it is a direct command to trigger the arithmetic logic unit (ALU).

3.2 Assembly: The Human Mirror

Assembly language provides a human-readable text representation of bytecode. It maintains a strict 1:1 mapping. Every line of assembly corresponds to exactly one or more bytes in the final binary. This allows for absolute control without the cognitive load of raw hex editing.

The Transformation Lifecycle

```
; The transformation from logic to binary
Sage:  print x + y
Asm:   GET_GLOBAL "x"; GET_GLOBAL "y"; ADD; PRINT
Binary: 05 00 01 05 00 02 0F 2A
```

4. Architectural Patterns: Stack vs. Register

SageVM is a dual-architecture machine. It supports both the SVM (Stack) and SRVM (Register) models. Understanding their differences is the key to low-level optimization.

4.1 The Stack machine (SVM)

An SVM operates on a Last-In, First-Out (LIFO) operand stack. All operations happen at the top. To add two numbers, you "push" them both onto the stack. The "ADD" instruction pops them, adds them, and pushes the result back.

- **Density:** Instructions are tiny (often 1 byte) because they don't need to specify where the data is.
- **Ease of Compilation:** Compilers naturally generate stack-based code from tree-like structures (ASTs).

4.2 The Register Machine (SRVM)

An SRVM operates on a fixed set of registers (bins). Each instruction explicitly names its source and destination (e.g., "add x10, x11, x12").

- **Performance:** Eliminates "stack traffic." Modern CPUs can execute register logic significantly faster.
- **JIT-Friendliness:** Register VM instructions map 1:1 to physical registers on x86, ARM, and RISC-V.

5. Sage SVM: The Stack Engine

SVM is the workhorse of the Sage ecosystem. It is the primary target for the standard Sage compiler.

5.1 The Operand Stack

The stack is an array of dynamically-typed Sage values. Every operation must maintain "Stack Neutrality"—ensuring the stack depth remains predictable to prevent overflows.

```
; Basic Stack Flow
CONSTANT 10 ; Stack: [10]
CONSTANT 20 ; Stack: [10, 20]
ADD         ; Stack: [30]
POP         ; Stack: []
```

5.2 Scope and Environments

SVM handles variables through "Environments." A `PUSH_ENV` instruction creates a new lexical scope. This is how local variables in functions are isolated from the global namespace.

6. Sage SRVM: The Register Engine

SRVM is the high-performance alternative. It is explicitly mapped to the ****RV64I (RISC-V)**** specification, allowing Sage code to run with near-native efficiency.

6.1 Register File (x0 - x31)

The SRVM provides 32 general-purpose 64-bit registers. Following the RISC-V ABI is critical for consistency.

Reg	Name	Role
x0	zero	Hardwired to 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x10-x17	a0-a7	Arguments / Returns

6.2 The VMSYS Extension

Because RISC-V is a general-purpose spec, it has no concept of "Sage Objects" or "Global Print." SageVM adds the **VMSYS** instruction, which uses the standard **SYSTEM** opcode to trigger VM services.

7. The Compilation Pipeline

A program travels through several layers of abstraction before it reaches the VM execution loop.

1. Source (.sage): Human-readable high-level code.
2. AST: The compiler parses source into a logical tree of operations.
3. Assembly (.svm): The AST is "flattened" into a linear series of mnemonics.
4. Linking (sgvmc): The assembler collects chunks and produces a binary.
5. Binary (.sgvm): The final, executable artifact.

```
User Logic [Lexer/Parser] AST [Emitter] Assembly
                                     |
Execution [MetalVM Loop] .sgvm Binary [sgvmc]
```


8. Binary Internals: The SGVM Format

Every executable must live in a standardized container. The SGVM format is designed for portability and fast loading.

8.1 Header Specification

Offset	Size	Field	Value
0x00	4 bytes	Magic	"SGVM"
0x04	2 bytes	Version	Major/Minor (0.1)
0x06	2 bytes	Functions	Count of func chunks
0x08	2 bytes	Constants	Size of pool

8.2 The Constant Pool

To save memory, all strings and numbers are stored in a centralized pool. Instructions refer to these via 2-byte indices. This "String Pooling" significantly reduces the binary footprint.

9. SGVM vs. ELF: A Technical Comparison

In standard Linux systems, compilers produce ELF files. Why does SageVM use its own SGVM format?

9.1 The Case Against ELF for VMs

ELF (Executable and Linkable Format) was built for silicon. It expects native machine code (x86, ARM). This creates three major problems for virtual machines:

- **Architectural Lock-in:** An ELF compiled for x86 won't run on ARM. SGVM is bytecode; it runs anywhere.
- **Object Isolation:** ELF has no concept of "Garbage Collection" or "Class Tables." SGVM carries the metadata required for the VM to manage object lifecycles.
- **Capability-Based Security:** SGVM allows instructions to be tagged with permissions (e.g., "Allow GPU", "Deny Filesystem"). ELF is "all-or-nothing" security.

10. Systems Engineering: Bootloaders & Kernels

In the Sage ecosystem, the SRVM acts as the "Silicon" layer for the OS. A bootloader is the first code to execute.

10.1 The SRVM Bootloader Pattern

```
; --- Kernel Bootloader Logic ---  
ldc x10, 0x00      ; Index 0: Memory Heap  
mv x3, x10         ; Set Global Pointer  
ldc x10, 0x01      ; Index 1: Stack Memory  
mv x2, x10         ; Set Stack Pointer  
vmsys x0, 0x02     ; Initialize VM Isolation (PUSH_ENV)  
ldc x10, 0x02      ; Index 2: Kernel Entry Point  
vmsys x10, 0x04    ; Transfer Control (VM0_CALL)  
vmsys x0, 0x01     ; VM_HALT if kernel crashes
```

11. Advanced Graphics: The Vulkan Bridge

SageVM treats high-performance graphics as a first-class citizen. 28 dedicated opcodes (59-86) provide direct access to the host GPU.

11.1 The Async Command Stream

Graphics aren't "drawn" instantly. They are recorded into a command buffer and submitted to the hardware. This prevents the CPU from waiting for the GPU.

```
; Record GPU Commands
OP_GPU_POLL_EVENTS
CONSTANT 0; OP_GPU_ACQUIRE_IMG
DEFINE_GLOBAL "idx"
CONSTANT 1; OP_GPU_BEGIN_COMMANDS
GET_GLOBAL "idx"
CONSTANT 2; OP_GPU_CMD_BEGIN_RP      ; Begin Render Pass
CONSTANT 3; OP_GPU_CMD_BIND_GP      ; Bind Graphics Pipeline
OP_GPU_CMD_DRAW                      ; Execute Draw Call
OP_GPU_CMD_END_RP
OP_GPU_END_COMMANDS
OP_GPU_PRESENT                        ; Finalize Frame
```

12. Object-Oriented Assembly Mechanics

Unlike most VMs, SageVM understands classes, methods, and inheritance at the opcode level.

12.1 The Method Table

In SVM assembly, a class is defined by a series of method bindings. When a method is called, the VM performs an optimized lookup in the class table.

```
; OO Implementation
CLASS "Animal"
METHOD "speak"          ; Bind speak logic
CLASS "Dog"
GET_GLOBAL "Animal"
INHERIT                 ; Clone "speak" behavior
METHOD "bark"           ; Add dog-specific method
```

13. Exception Handling & Runtime Safety

SageVM ensures system stability via a robust, low-overhead exception handling mechanism.

13.1 The Handler Stack

Entering a "try" block pushes a recovery record onto a hidden handler stack. This record saves the state of the machine at the point of entry.

Exception Flow Pattern

```
SETUP_TRY PANIC_LABEL
CONSTANT "Dangerous Operation"
RAISE
END_TRY
JUMP EXIT
PANIC_LABEL:
    PRINT
EXIT:
```

14. Optimization for Assembly Writers

In a virtual world, efficiency is measured in instruction count and memory traffic.

14.1 Traffic Reduction

Repeatedly calling `GET_GLOBAL` is expensive (dictionary lookup). Use the `DUP` (Duplicate) opcode to reuse values already on the stack.

14.2 Register Pressure in SRVM

When writing for the register machine, utilize "Saved Registers" (`s0-s11`). These are guaranteed to be preserved across function calls, reducing the need for costly stack spills.

15. The Future: Path to v1.0 and Beyond

SageVM is a living architecture. Our roadmap focuses on closing the gap between virtual and native execution.

15.1 Just-In-Time (JIT) Compilation

The next major release of MetaVM will include a background JIT. It will monitor "hot" chunks of assembly and compile them into native x86 or ARM machine code on the fly.

End of Document · Definitive SageVM Assembly Guide v0.9.8
A Publication of Night-Traders-Dev Systems Group